

Gérer les accès à une API

Table des matières

I. S'authentifier à l'aide d'un mot de passe	3
II. Exercice : Quiz	7
III. S'authentifier à l'aide d'un token	7
IV. Exercice : Quiz	13
V. OAuth : protocole d'authentification	14
VI. Exercice : Quiz	21
VII. Essentiel	21
VIII. Auto-évaluation	22
A. Exercice :	22
B. Test	22
Solutions des exercices	24

I. S'authentifier à l'aide d'un mot de passe

Durée : 1 h

Environnement de travail : windows / (Mac OSX)

Pré-requis :

- Savoir développer une API REST
- Bases de Node.js
- Bases d'Express.js

Contexte

Les applications modernes sont souvent développées autour d'une API REST et différents clients (web, mobile, desktop) qui s'authentifient et communiquent avec elles.

Dans ce cours, nous allons explorer, à l'aide de Node.js et d'Express.js, les méthodes d'authentification les plus populaires et utilisées.

D'abord, vous apprendrez à implémenter un système d'authentification simple appelé HTTP Basic.

Ensuite, vous découvrirez le standard JSON Web Token et apprendrez à l'implémenter et à le consommer côté client.

Enfin, nous concluons ce cours par une introduction à OAuth : ce protocole n'est pas tout à fait une méthode d'authentification, mais il est souvent utilisé comme tel.

Pour suivre ce cours et faire les exercices, il vous faudra télécharger une API REST disponible gratuitement sur GitHub à cette adresse¹ : téléchargez le projet sur votre machine et suivez les instructions notées dans le fichier *README.md*.

À la fin de ce cours, vous serez en capacité de mettre en place un système d'authentification sur vos API REST et de vous authentifier à des API REST fermées.

Objectifs de la partie

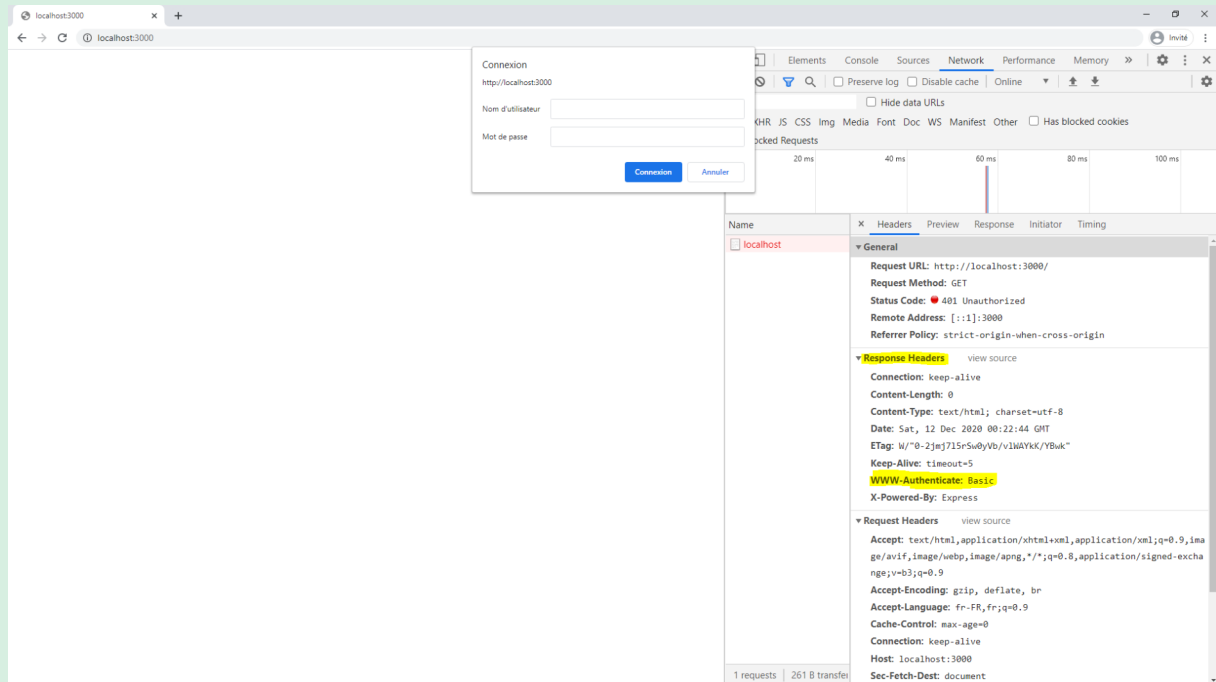
- Expliquer l'authentification HTTP Basic
- Implémenter côté serveur
- Vérifier côté client

Contexte

Lorsqu'on pense à l'authentification, on pense généralement à un formulaire avec un login et un mot de passe. Lorsque nous n'avons pas la possibilité ou la volonté d'afficher un formulaire d'authentification, nous pouvons utiliser le mécanisme HTTP « Basic » qui permet à un client HTTP d'envoyer au serveur un login et un mot de passe encodé.

¹ <https://github.com/mickaelandrieu/bookstore-node-rest-api>

Quand le client va faire une première demande au serveur REST, le serveur va lui retourner une réponse vide avec le code 401 accompagné d'un en-tête « *WWW-Authenticate Basic* ». La plupart des navigateurs vont afficher alors un formulaire de connexion s'ils détectent cet en-tête HTTP :



Une fois les informations soumises et validées, le serveur nous autorise l'accès.

D'un point de vue HTTP, voici ce qu'il se passe entre le serveur (notre API REST) et le client HTTP (notre application web ou mobile) :



Dans cette partie, nous allons voir comment en quelques lignes de code nous pouvons mettre en place ce système d'authentification et comment côté client nous allons pouvoir nous authentifier auprès de l'API REST.

Méthode

Comme souvent dans l'écosystème Node.js et Express.js, des développeurs et développeuses ont eu à implémenter cette fonctionnalité avant vous et ont créé une librairie pour cela !

Il faudra installer « *express-basic-auth* » dans votre projet :

npm i express-basic-auth

Cette librairie se comporte comme une extension du framework Express.js que nous allons activer.

Nous importons ensuite la librairie et nous la configurons :

```
1 const app = express()
2 const basicAuth = require('express-basic-auth')
3
4 app.use(basicAuth({
5   users: {
6     'john': 'superSecret',
7     'elise': '12345678',
8   },
9   challenge: true
10}))
11
12 app.get('/', (req, res) => {
13   res.json({'message': "Home, nothing interesting here."})
14 })
15
16 app.listen(3000, () => {
17   console.log('[Server] Started.')
18 })
```

Maintenant, si vous exécutez le fichier et que vous accédez au navigateur sur le port 3 000, vous devriez voir le formulaire apparaître comme dans l'une des captures d'écran précédentes.

Vous pouvez alors entrer les identifiants qui sont proposés dans ce code d'exemple : l'utilisateur « **john** » qui a pour mot de passe « **superSecret** ».

Revenons un peu au code produit

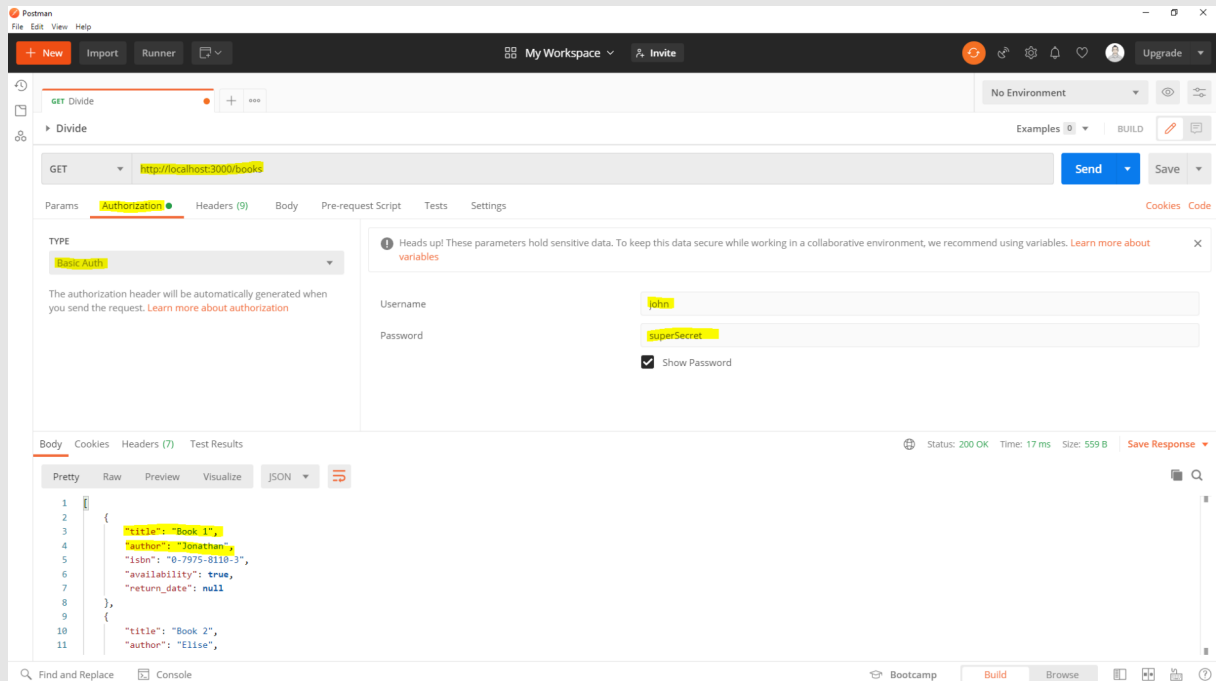
- D'abord, nous avons inclus l'extension Express avec un **require**(« *express-basic-auth* »),
- Ensuite, nous avons configuré et activé cette extension avec le code suivant :

```
1 app.use(basicAuth({
2   users: {
3     'john': 'superSecret',
4     'elise': '12345678',
5   },
6   challenge: true
7}))
```

Comme on peut le voir, l'objet **basicAuth** a 2 propriétés :

1. La propriété **users** qui est une liste de login/mot de passe, pour un nouvel utilisateur il faudra ajouter ici une nouvelle entrée,
2. La propriété **challenge** permet d'ajouter l'en-tête HTTP « *WWW-Authenticate Basic* » dans la réponse de l'application. Si cet en-tête n'est pas présent, alors le navigateur n'affiche pas de formulaire de connexion.

Comment vérifier que cette API fonctionne sans écrire de code ? Nous pouvons par exemple utiliser le logiciel Postman¹ :



Et voilà !

N'hésitez pas à utiliser cette API REST de test, disponible à cette adresse, pour expérimenter la notion d'authentification²

Exemple Code

```
1 // index.js
2 const app = express()
3 const basicAuth = require('express-basic-auth')
4
5 app.use(basicAuth({
6   users: {
7     'john': 'superSecret',
8     'elise': '12345678',
9   }
10 })))
```

Syntaxe À retenir

Implémenter l'authentification **HTTP Basic** côté serveur revient à attendre des applications clientes qu'elles envoient une requête avec une entête HTTP **Authorization**, avec la valeur **Basic** suivi des identifiants de connexion qui auront été encodés pour des questions de sécurité.

Il existe de nombreuses bibliothèques compatibles avec Express.js pour implémenter ce système en quelques lignes de code, et pour un projet de test c'est plutôt pratique et rapide de s'authentifier côté client, que ce soit par le navigateur ou à l'aide d'un logiciel comme Postman.

¹ <https://www.postman.com/>

² <https://bookstore-http-basic-auth.herokuapp.com/>

Mais dans la pratique, ce système d'authentification n'est pas vraiment utilisé ou recommandé en production. On lui préférera le standard JWT que vous allez découvrir dans la prochaine partie.

Complément

- HTTP Basic (explication complète)¹
- Basic Auth Express² : plugin HTTP Basic pour Express.js

II. Exercice : Quiz

Question

Il est temps de mettre en place l'authentification HTTP sur le projet que l'on vous a demandé de télécharger en introduction de ce cours.

Faites en sorte que 2 utilisateurs puissent avoir accès à l'API REST :

1. L'utilisateur jonathan@studi.com avec le mot de passe « *sup3rS3cr3t* »,
2. Et l'utilisatrice ieloushana@studi.com avec le mot de passe « *YouKn0wN0thingJhon* ».

Indice :

Vous allez devoir installer et configurer le projet en utilisant les instructions contenues dans le fichier README.md, puis installer la librairie que nous venons de voir et ajouter les instructions.

III. S'authentifier à l'aide d'un token

Objectifs de la partie

- Comprendre ce qu'est le standard JSON Web Token
- Implémenter côté serveur
- Implémenter côté client

Contexte

Dans le chapitre précédent, vous avez appris comment reproduire le comportement d'un navigateur à l'aide d'un login et d'un mot de passe.

L'essentiel des API REST utilisent plutôt une autre méthode d'authentification à l'aide d'un token.

Un token, c'est une chaîne encodée qui contient plusieurs informations qu'il vous semblera utile de partager avec les utilisateurs de votre API.

Par exemple, après authentification ce token pourrait contenir les informations du profil d'un utilisateur, ou le nombre de requêtes qu'il est autorisé à faire sur l'API ou encore jusqu'à quand le token est valide.

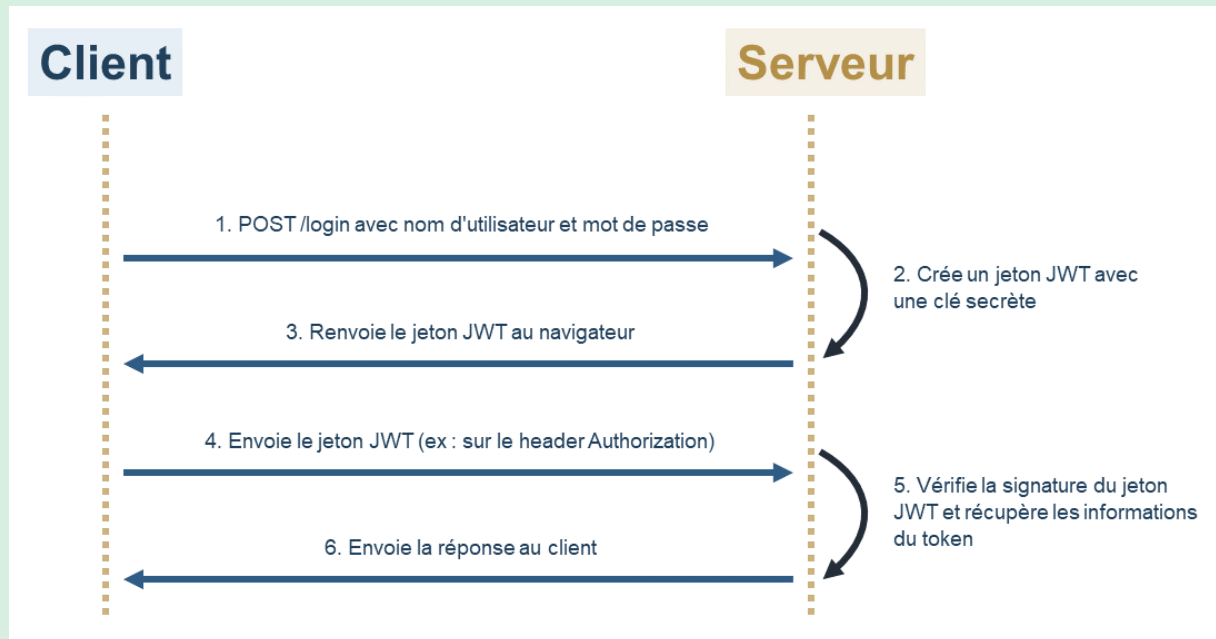
JWT³ (« *JSON Web Token* ») est un standard qui définit comment vous devriez concevoir vos tokens pour partager en toute sécurité des informations avec vos utilisateurs.

¹ <https://developer.mozilla.org/fr/docs/Web/HTTP/Authentication>

² <https://github.com/LionC/express-basic-auth>

³ <https://jwt.io/>

Le fonctionnement général sur la relation Client/Serveur pour une authentification JWT est décrit dans le schéma ci-dessous :



Comment est construit un JSON Web Token ?

Un token (ou « *jeton* » en français) est une simple chaîne de caractères composée de 3 parties : un **en-tête** (« *header* »), une **enveloppe** (« *payload* ») et une **signature** (« *signature* »).

L'en-tête

Pour implémenter le standard JWT, l'en-tête doit contenir 2 informations :

- L'algorithme d'encodage : définit la façon dont vont être encodées les informations (pour une question de sécurité),
- Le type de token : ici ce sera toujours « **JWT** ».

Par exemple, voici à quoi ressemble un en-tête de token JWT :

```

1 {
2   "alg": "HS256",
3   "typ": "JWT"
4 }
  
```

Ensuite, ce JSON sera encodé à l'aide de Base64Url¹ pour ressembler à ça :

ewogICJhbGciOiAiSFMyNTYiLAogICJ0eXAiOiAiSldUIgp9

L'enveloppe ou le payload

La seconde partie du token est appelée enveloppe ou payload, et elle contient tout ce qui pourrait être utile pour l'utilisateur. S'il n'y a rien d'obligatoire dans ce JSON, par convention on notera **sub** le sujet du token et **exp** la date d'expiration en secondes depuis le 1^{er} janvier 1970 (l'epoch²).

Voici la liste de toutes les propriétés³ (ou encore « *claims* », dans le contexte) de la spécification JWT.

¹ <http://www.base64url.com/login>

² <https://fr.wikipedia.org/wiki/Epoch>

³ <https://www.iana.org/assignments/jwt/jwt.xhtml>

Ainsi, un exemple valide d'un payload pourrait avoir cette forme :

```
1 {
2   "sub": "authentication",
3   "name": "Jonathan Doe",
4   "admin": true,
5   "exp": 1894567890
6 }
```

Comme précédemment, ce JSON sera encodé en Base64 et ressemble à ça :

```
ewogICJzdWIiOiAiYXV0aGVudGlmaWNhdGlvbiiIsCiAgIm5hbWUiOiAiSm9uYXRoYW4gRG91IiwKICAiYWRTaW4iOiB0cnVlLAogICJleHAiOiAxODk0NTY3ODkwCn0=
```

La signature

Pour créer la signature, nous avons besoin de l'en-tête encodé, du payload encodé, d'une clé secrète et de l'algorithme que nous avons défini dans l'en-tête.

Le principe ressemblerait à l'appel d'une fonction qui retournerait le token :

```
1 secret = "my-wonderful-rest-api"
2 HMACSHA256(
3   base64UrlEncode(header) + "." +
4   base64UrlEncode(payload),
5   secret)
```

Vous apprendrez dans la section « Méthode » comment implémenter ceci en Node.js.

Pour le résultat final suivant :

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJhdXRoZW50aWZpY2F0aW9uIiwiaWF0IjE4OTQ1Njc4OTB9.hvnTcVcZRKkT7nnNPSNIGqunASrAEr7QEkYmFVTA
```

Envie d'expérimenter avant de mettre les mains dans l'implémentation ? **Essayez le débogueur fourni par JWT¹**. Voici ce que vous obtiendrez en entrant toutes les informations de l'exemple sur leur interface :

Algorithm HS256

Encoded

PASTE A TOKEN HERE

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJhdXRoZW50aWZpY2F0aW9uIiwibmFtZSI6IkpvaW50aWZpY2F0aW9uIiwiaWF0IjE4OTQ1Njc4OTB9.hvn-TcVcZRRkT7nnNPSN1Gq-_unASrAer7QEkYmFVTA
```

Decoded

EDIT THE PAYLOAD AND SECRET

HEADER: ALGORITHM & TOKEN TYPE

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

PAYLOAD: DATA

```
{
  "sub": "authentication",
  "name": "Jonathan Doe",
  "admin": true,
  "exp": 1894567890
}
```

VERIFY SIGNATURE

```
HMACSHA256(
  base64UrlEncode(header) + "." +
  base64UrlEncode(payload),
  your-256my-wonderful-
) ☐ secret base64 encoded
```

Signature Verified

SHARE JWT

Méthode

Il est temps de passer à la pratique ! Pour implémenter une authentification JWT avec Node.js, il faudra installer les librairies **jsonwebtoken²** et **express-jwt³** dans un projet Node.js.

npm i jsonwebtoken && npm i express-jwt

Vous devrez cette fois vous authentifier à l'aide d'un point d'entrée conçu pour cela, de sorte à obtenir le token qui vous permettra **ensuite d'accéder à la partie sécurisée** de l'API.

Pour faire simple, considérons une API REST avec 2 points d'entrée (ou « *endpoint* ») :

- **/login** sera l'accès pour s'authentifier à l'aide d'un login et d'un mot de passe,
- **/private-access** qui retournera un résultat uniquement si la personne est authentifiée.

Le principe est le suivant, l'utilisateur (ou le programme) doit d'abord obtenir le token auprès de **/login** avant de pouvoir accéder à **/private-access**.

S'il ne fournit pas de token, ou que celui-ci est invalide, alors vous retournerez un code d'erreur **401 Unauthorized⁴**.

¹ <https://jwt.io/#debugger-io>

² <https://www.npmjs.com/package/jsonwebtoken>

³ <https://www.npmjs.com/package/express-jwt>

⁴ <https://developer.mozilla.org/fr/docs/Web/HTTP/Status/401>

10

Tous droits réservés à STUDI - Reproduction interdite

Sécuriser l'accès à l'URI /private-access

La librairie **express-jwt** vous permet en quelques lignes de protéger tous les points d'entrée de l'API REST créée :

```
1 const e_jwt = require('express-jwt');
2
3 app.use(e_jwt( { secret: SECRET, algorithms: ['HS256']} ).unless( {path: ['/login']} ) ) );
```

Vous utilisez ici une clé secrète pour pouvoir vérifier la signature du token, et l'algorithme utilisé (HS256). Mais vous devez aussi autoriser au *minimum* la page de connexion, de sorte que l'on puisse s'authentifier et obtenir un token.

Page de connexion

```
1 const jwt = require('jsonwebtoken')
2 const SECRET = 'mon-application-rest'
3
4 app.post('/login', (req, res) => {
5   if (!req.body.username || !req.body.password) {
6     return res.status(400).json({ message: 'Error. Please enter the correct username and
7 password' })
8   }
9   const user = users.find(u => u.username === req.body.username && u.password ===
10 req.body.password)
11   if (!user) {
12     return res.status(400).json({ message: 'Error. Wrong login or password' })
13   }
14   const token = jwt.sign({
15     username: user.username,
16     role: user.role
17   }, SECRET, { expiresIn: '3 hours' })
18   return res.json({ access_token: token })
19 })
```

Ici, à partir des informations reçues de la requête de l'utilisateur, on autorise la création d'un token à l'aide de la fonction **sign()** de la librairie **jsonwebtoken**.

Cette fonction accepte de nombreuses options :

- Tout d'abord, les éléments du payload que le token va retourner : ici, le **username** et le **rôle** de l'utilisateur identifié,
- Ensuite, pour générer le token vous avez besoin de la clé secrète de signature ! Par défaut la librairie utilise l'encodage HS256 c'est pourquoi vous n'avez pas besoin de le définir,
- Enfin, vous définissez **une durée de validité** du token : c'est une mesure de sécurité très importante.

Exemple Code

```
1 # index.js
2 const express = require('express');
3 const app = express()
4 const jwt = require('jsonwebtoken')
5 const e_jwt = require('express-jwt');
6
7 const SECRET = 'S3cr3tK3yD0ntSh4r31t'
8
9 app.use(express.json())
```

```

10
11 const users = [
12   {
13     "username": "john_doe",
14     "password": "123456",
15     "role": "USER"
16   },
17   {
18     "username": "jane_doe",
19     "password": "123456",
20     "role": "ADMIN"
21   }
22 ]
23
24 app.use(e_jwt({ secret: SECRET, algorithms: ['HS256']})).unless({path: ['/login', '/']}));
25
26 app.use(function (err, req, res, next) {
27   if (err.name === 'UnauthorizedError') {
28     res.status(401).send('Invalid token, access /login with username and password in the
Body to authenticate.');
```

```

65 {
66   "name": "jwt-exemple",
67   "version": "1.0.0",
68   "description": "",
69   "main": "index.js",
70   "scripts": {
71     "start": "node index.js"
72   },
73   "author": "",
74   "license": "ISC",
75   "dependencies": {
76     "express": "^4.17.1",
77     "express-jwt": "^6.0.0",
78     "jsonwebtoken": "^8.5.1",
79     "jwt-express": "^1.1.0",
80     "moment": "^2.29.1"
81   }
82 }

```

Vous découvrirez comment vous connecter à cette **API REST** dans la vidéo de démonstration.

Syntaxe À retenir

JWT définit la structure d'un token composé de 3 parties principales :

- Un **en-tête** qui contient le type de token (JWT) et l'algorithme choisi pour encoder les données envoyées aux clients de l'API,
- Une **enveloppe (ou encore « payload »)** qui contient les informations utiles aux utilisateurs de l'API. Bien que certaines propriétés soient recommandées par le standard JWT elles ne sont pas obligatoires,
- Une **signature**, qui reprend les deux premiers éléments et encode tout le contenu à l'aide de l'algorithme choisi dans l'en-tête.

Pour l'implémentation, quel que soit votre langage de programmation favori (PHP, Node.js, Python, Java, etc.) il existe de très bonnes bibliothèques pour faciliter la mise en place de l'authentification JWT.

Pour Node.js, vous pouvez utiliser les bibliothèques **jsonwebtoken** et **express-jwt**.

Complément

- <https://jwt.io/> (Documentation, Guide et débogueur)
- **JWT en PHP**¹ avec API Platform et Symfony
- **JWT à l'aide de Python**²

IV. Exercice : Quiz

Question

Vous vous rappelez le projet que vous avez téléchargé et sur lequel vous aviez mis l'authentification HTTP Basic ? Cette fois, modifiez-le pour mettre en place l'authentification JWT.

Il est recommandé de faire une copie du projet original pour pouvoir, par la suite, comparer facilement les deux méthodes d'authentification.

¹ <https://api-platform.com/docs/core/jwt/>

² <https://pyjwt.readthedocs.io/en/stable/>

Indice :

Vous allez devoir installer les deux librairies, créer un nouveau point d'entrée pour permettre l'authentification et sécuriser le reste à l'aide de la librairie **express-jwt**. N'hésitez pas à consulter la vidéo de démonstration précédente.

VI. OAuth : protocole d'authentification

Objectifs de la partie

- Comprendre la différence entre authentification et autorisation
- Comprendre comment fonctionne OAuth
- Être capable de mettre en place une authentification/autorisation via OAuth

Contexte

Et si le meilleur système d'authentification était de ne pas en avoir du tout ? Dans les chapitres précédents, nous avons nécessairement des utilisateurs et un moyen d'identification. Cela implique que vos utilisateurs doivent préalablement être « inscrits » à votre API pour pouvoir ensuite être identifiés et l'utiliser.

En utilisant OAuth, et en fournissant plusieurs fournisseurs d'authentification comme « Facebook Login » ou « Google Sign In », vous n'aurez plus besoin de gérer des utilisateurs. Le gros intérêt ici est que si une personne malveillante a accès à vos données, elle ne pourra pas identifier vos utilisateurs !

Pour cela, vous devrez vous enregistrer auprès de ces fournisseurs : Facebook, Google, Twitter, LinkedIn parmi les nombreux disponibles.

Vous allez déclarer votre application auprès de ces entreprises pour qu'elles autorisent votre API REST à demander des informations sur « leurs » utilisateurs.

C'est en ce sens qu'OAuth n'est pas un système d'identification mais d'autorisation : ce protocole définit un ensemble de règles pour demander des informations à des applications.

C'est donc un système qui définit une interaction non pas avec un seul serveur, mais avec deux : le serveur de notre application, mais également le serveur d'authentification (Facebook, par exemple).

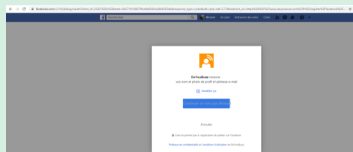
Dans ce chapitre, nous allons implémenter un système d'authentification Facebook, ce qui est également appelé un « login social ». Vous ne demanderez que très peu d'informations à Facebook pour accepter l'accès à votre API REST, mais elles seront suffisantes pour que vous puissiez ensuite identifier vos utilisateurs.

Nous allons, encore une fois, utiliser des librairies de l'écosystème Node.js et notamment **Passport**¹ qui permet l'authentification à plus de 500 fournisseurs d'authentification !

Comment ça fonctionne ?

Il y a 3 étapes pour mettre en place une authentification OAuth 2 :

- Votre application demande au fournisseur des autorisations : en général ça provoque l'ouverture d'un formulaire de connexion chez le fournisseur, qui permet à l'utilisateur d'entrer ses informations de connexion.



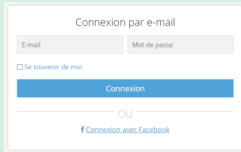
- Une fois authentifié auprès du service, le fournisseur fait une redirection de l'utilisateur vers votre application en lui envoyant un token.

¹ <http://www.passportjs.org/>

- À l'aide de ce token, votre application peut faire une requête auprès du fournisseur pour obtenir des informations : **vous devez donc stocker ce token.**

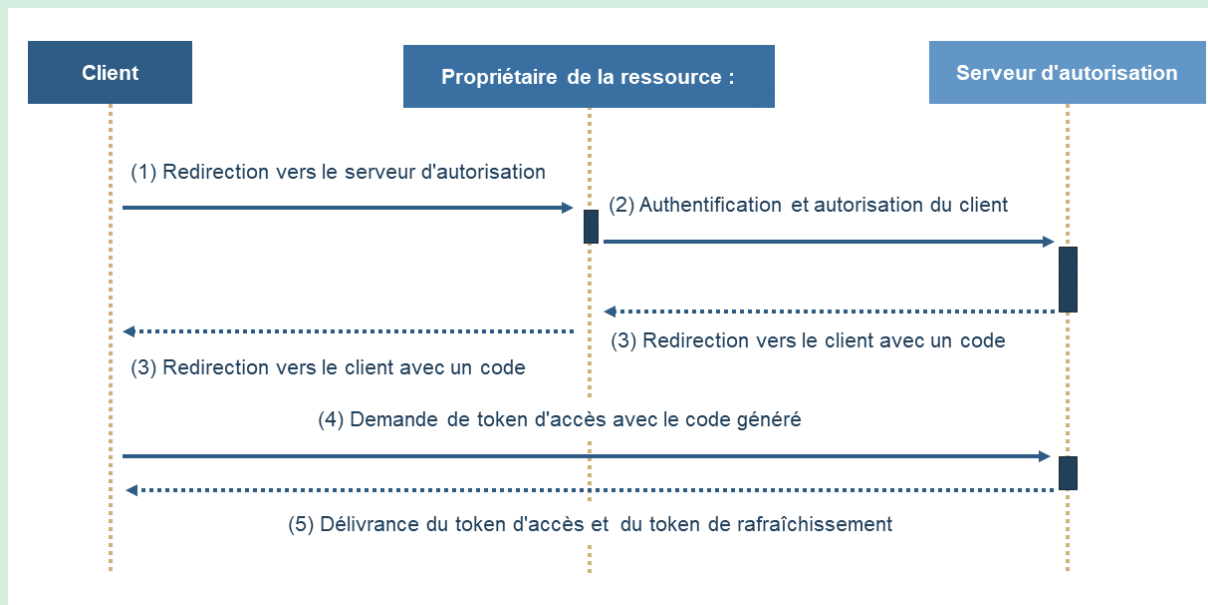
Pour implémenter un système d'authentification OAuth 2, vous aurez donc besoin de 3 points d'entrées (ou routes) :

- Un point d'entrée capable d'afficher une interface pour afficher le formulaire de connexion au fournisseur (dans notre cas, Facebook), (**/login**).



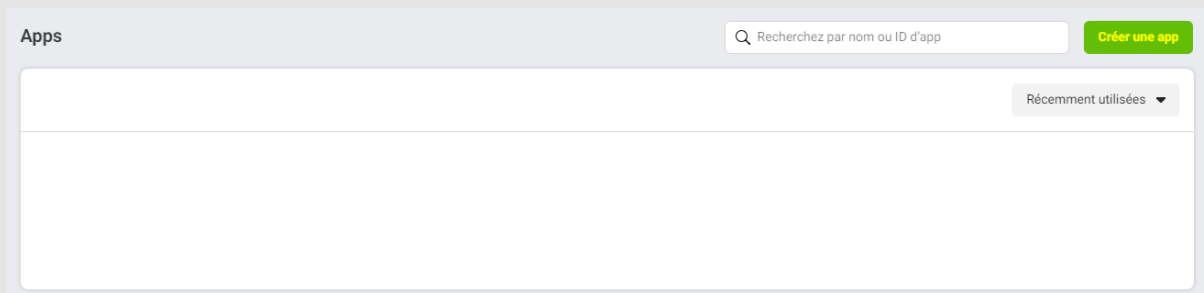
- Un point d'entrée que **le fournisseur utilise pour vous communiquer le token** (et c'est d'ailleurs pour cela que vous avez besoin d'enregistrer votre application : le fournisseur a besoin de cette information), (**/oauth-redirect**).
- Au minimum pour que ça soit intéressant, un point d'entrée qui utilise ce token pour récupérer des informations auprès de ce fournisseur, **pour identifier l'utilisateur dans votre application**, (**/action**).

Ceci pourrait être résumé par le schéma suivant :



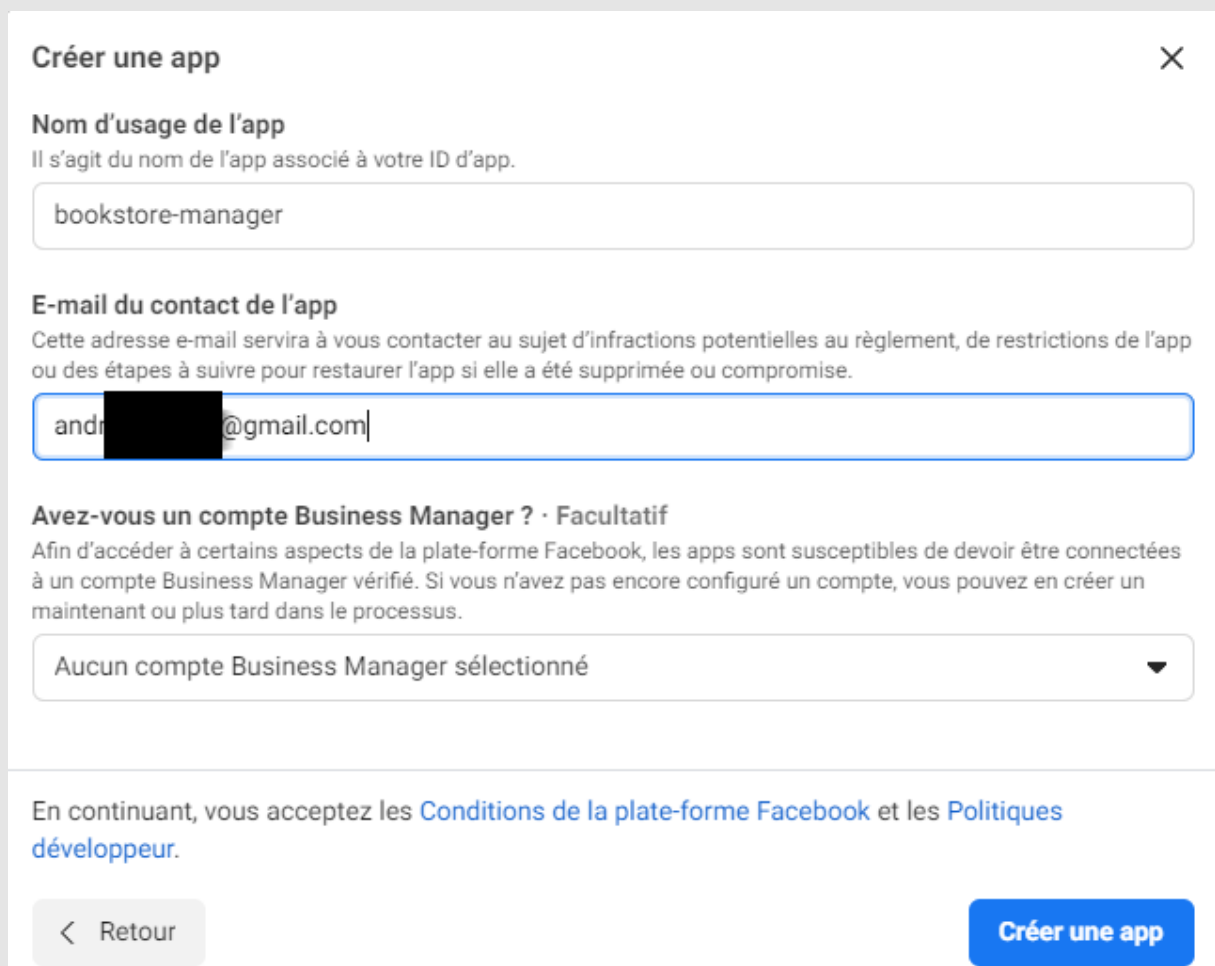
Méthode

Tout d'abord, il va falloir créer un compte **Facebook Developers**¹ et déclarer une nouvelle application dans l'onglet « **Apps**² » :



Cliquez sur « *Créer une app* » et dans les choix proposés : « *Créer des expériences partagées* ».

Un formulaire s'ouvre et vous demande d'entrer le nom de l'application et un email de contact :



Une nouvelle fenêtre s'ouvre, avec une proposition des différents services que propose Facebook !

Vous allez utiliser le service Facebook Login en cliquant sur le bouton « *Configurer* », puis « *Web* ».

¹ <https://developers.facebook.com/>

² <https://developers.facebook.com/>

Facebook vous demande alors de mettre l'URI de votre application et l'URI qu'il doit utiliser pour vous renvoyer le token (appelée « **URI de Callback** ou l'**URI de redirection** ») :


Ajoutez facilement Facebook Login à votre app avec notre [Quickstart](#)

Paramètres OAuth client

☒ **Oui**

Connexion OAuth client
Active le processus de token client OAuth standard. Sécurisez votre app et empêchez tout abus en verrouillant les URI de redirection de token autorisées avec les options ci-dessous. Désactivez-les complètement si vous ne les utilisez pas. [?]

☒ **Oui**

Connexion OAuth web
Active la connexion client OAuth web. [?]

☐ **Non**

Forcer la ré-authentification OAuth web
Une fois activée, invite les gens à entrer leur mot de passe Facebook avant de se connecter sur le web. [?]

☒ **Oui**

Utiliser le mode strict pour les URI de redirection
N'autorise que les redirections qui utilisent le SDK Facebook ou qui correspondent exactement aux URI de redirection OAuth valides. Hautement recommandé. [?]

☒ **Oui**

Imposer le HTTPS
Imposez l'utilisation du HTTPS pour les URI de redirection et le SDK JavaScript. Vivement recommandé. [?]

☐ **Non**

Connexion OAuth de navigateur intégrée
Activez les URI de redirection de webview pour la connexion client OAuth. [?]

URI de redirection OAuth valides

☐ **Non**

Connexion à partir des appareils
Activer le processus de connexion client OAuth pour les appareils tels que les smart TV [?]

Ensuite, vous pouvez récupérer un ID et une clé secrète (**SECRET_KEY**) liée à l'application, qui seront utilisés par l'application que vous allez mettre en place. Pour cela, cliquez sur « *Paramètres* », puis « *Général* ».

Identifiant de l'application

Clé secrète

Nom d'usage

Namespace

Domaines de l'app

Adresse e-mail de contact ⓘ

URL de la Politique de confidentialité

URL des conditions de service

Suppression des données utilisateur ⓘ

Icône de l'app (1024 x 1024)


1024 x 1024

Catégorie

En savoir plus sur les catégories d'app [ici](#)

But de l'app
Le but principal de cette app est d'accéder aux données de la plate-forme Facebook et de les utiliser au nom de :
☐ Vous-même ou votre propre entreprise
☐ Clients ⓘ
Si vous développez une app qui accède et utilise des données de la plate-forme Facebook pour le compte de clients, vous êtes soumis à la [Section 5b des conditions générales de la plate-forme](#).

Développement du système d'authentification

Tout d'abord, vous avez besoin de stocker de l'information et notamment le jeton retourné par Facebook. Pour cela vous utiliserez le **système de session**¹ du framework Express.

Il s'active et se configure de cette façon :

```
1 const session = require('express-session');
2 app.use(session({
3   resave: false,
4   saveUninitialized: true,
5   secret: 'ma-super-app'
6 }));
```

Un utilisateur aura la même session entre plusieurs appels à l'API, c'est donc un système adapté à votre besoin.

Ensuite, la librairie Passport et le plugin pour Facebook :

```
1 const passport = require('passport');
2 const FacebookStrategy = require('passport-facebook').Strategy;
3 const FACEBOOK_ID = 'put_your_own_fb_id'
4 const FACEBOOK_SECRET = 'put_your_fb_secret'
5
6 app.use(passport.initialize());
7 app.use(passport.session());
8
9 passport.serializeUser(function (user, cb) {
10   cb(null, user);
11 });
12
13 passport.deserializeUser(function (obj, cb) {
14   cb(null, obj);
15 });
16
17 passport.use(new FacebookStrategy({
18   clientID: FACEBOOK_ID,
19   clientSecret: FACEBOOK_SECRET,
20   callbackURL: 'http://localhost:3000/oauth-redirect'
21 }, function (accessToken, refreshToken, profile, done) {
22   return done(null, profile);
23 }
24 ));
```

Ce code mérite quelques explications !

Nous allons sauvegarder des informations en session, donc nous configurons Passport pour qu'il utilise la session comme système de stockage.

Nous avons besoin d'installer les librairies **passport**², **passport-facebook**³ et de les **configurer**⁴.

Les informations passées par Facebook sont stockées et extraites en session entre chaque demande des utilisateurs (on peut imaginer une session comme une base de données où toutes les informations sont stockées sous forme de chaîne de caractères).

C'est pourquoi nous avons des fonctions **serializeUser** et **deserializeUser**, tel qu'indiqué dans **la documentation de Passport**⁵.

¹ <https://www.npmjs.com/package/express-session>

² <https://www.npmjs.com/package/passport>

³ <https://www.npmjs.com/package/passport-facebook>

⁴ <http://www.passportjs.org/packages/passport-facebook/>

⁵ <http://www.passportjs.org/docs/configure/#sessions>

Enfin, les 2 actions principales pour mettre en place notre « *Workflow* » OAuth 2 :

```

1 app.get('/login', passport.authenticate('facebook', {
2   scope: ['public_profile']
3 }));
4
5 app.get('/oauth-redirect',
6   passport.authenticate('facebook', {
7     successRedirect: '/action',
8     failureRedirect: '/error'
9   })
10 );

```

D'abord, l'appel au `/login` qui déclenche l'authentification auprès de Facebook. La fonction `authenticate()` de la librairie permet de définir le scope des autorisations que l'on demande au réseau social.

Pour en savoir plus sur les autorisations possibles, consultez **la documentation¹** de Facebook.

Ensuite, Facebook va directement faire une requête sur `/oauth-redirect` :

- Si tout se passe bien, vous obtiendrez les informations et serez redirigés sur `/action`,
- S'il y a un souci avec l'authentification, vous serez redirigés vers `/error`.

En cas de succès, les informations envoyées par Facebook pourraient être stockées par votre application pour identifier vos utilisateurs *mais ce n'est pas obligatoire*.

Le but de ce chapitre n'est pas la maîtrise de Node.js, Express.js ou même de la librairie Passport mais la **bonne compréhension du protocole OAuth 2** par la mise en place d'un code fonctionnel. Si la librairie Passport vous intéresse, n'hésitez pas à parcourir en détail leur documentation et notamment les applications de démonstration.

Exemple Code

```

1 const express = require('express');
2 const app = express();
3 const session = require('express-session');
4 const passport = require('passport');
5 const FacebookStrategy = require('passport-facebook').Strategy;
6
7 const SECRET = 'S3cr3tK3yD0ntSh4r31t'
8 const FACEBOOK_ID = 'put_your_own_fb_id'
9 const FACEBOOK_SECRET = 'put_your_fb_secret'
10
11 app.use(express.json())
12 app.use(session({
13   resave: false,
14   saveUninitialized: true,
15   secret: SECRET
16 }));
17
18 app.use(passport.initialize());
19 app.use(passport.session());
20
21 passport.serializeUser(function (user, cb) {
22   cb(null, user);
23 });
24
25 passport.deserializeUser(function (obj, cb) {

```

¹ <https://developers.facebook.com/docs/permissions/reference>

```

26   cb(null, obj);
27 });
28
29 passport.use(new FacebookStrategy({
30   clientID: FACEBOOK_ID,
31   clientSecret: FACEBOOK_SECRET,
32   callbackURL: 'http://localhost:3000/oauth-redirect'
33 }, function (accessToken, refreshToken, profile, done) {
34   return done(null, profile);
35 }
36 ));
37
38 app.get('/login', passport.authenticate('facebook', {
39   scope: ['public_profile']
40 }));
41
42 app.get('/oauth-redirect',
43   passport.authenticate('facebook', {
44     successRedirect: '/action',
45     failureRedirect: '/error'
46   })
47 );
48
49 app.get('/action', isLoggedIn, function (req, res) {
50   res.json(req.user)
51 });
52
53 app.get('/error', isLoggedIn, function (req, res) {
54   res.send('Login error');
55 });
56 app.get('/logout', function (req, res) {
57   req.logout();
58   res.redirect('/');
59 });
60 function isLoggedIn(req, res, next) {
61   if (req.isAuthenticated())
62     return next();
63   res.redirect('/');
64 }
65
66 const port = process.env.PORT || 3000;
67 app.listen(port, () => {
68   console.log('[Minimal Sample Server] Started.')
69 })

```

```

1 {
2   "name": "oauth2-secure-api",
3   "version": "1.0.0",
4   "description": "",
5   "main": "index.js",
6   "scripts": {
7     "start": "node index.js"
8   },
9   "author": "Vous <prenom.nom@exemple.com>",
10  "license": "ISC",
11  "dependencies": {
12    "express": "^4.17.1",
13    "express-session": "^1.17.1",

```

```

14   "passport": "^0.4.1",
15   "passport-facebook": "^3.0.0"
16 }
17 }

```

Syntaxe À retenir

Le protocole OAuth 2 est plus compliqué à comprendre et à mettre en place. Heureusement, de très bonnes librairies existent et font l'essentiel.

Le plan pour mettre en place un système d'authentification OAuth 2 sera toujours le même :

- S'inscrire auprès du fournisseur d'authentification pour déclarer l'URI de votre application sur laquelle il doit renvoyer le token,
- Déclencher l'authentification chez le fournisseur, puis rediriger vers l'URI de votre choix en ayant obtenu les informations requises,
- Finalement, utiliser ces informations pour authentifier vos utilisateurs dans votre propre base de données ou alors demander des informations supplémentaires au fournisseur OAuth.

Complément

- La spécification OAuth 2¹
- L'extension **Passport Facebook**²
- L'**application de démonstration**³ de la Communauté Express

VII. Exercice : Quiz

Question

Cette fois, **pas de code à écrire** mais une authentification Facebook à **configurer et mettre en place**.

Téléchargez l'**application de démonstration**⁴ de la Communauté Express et prenez le temps de lire le code qu'elle contient.

Indice :

Le but du jeu ici consiste à réaliser 3 tâches :

- Créer une application sur Facebook,
- Obtenir les Facebook ID et Facebook Secret, et déclarer la « *bonne URI* » de callback,
- Exécuter l'instruction suivante dans le terminal pour vérifier que l'application fonctionne :

```

CLIENT_ID=__FACEBOOK_CLIENT_ID__CLIENT_SECRET=
__FACEBOOK_CLIENT_SECRET__ node server.js

```

VIII. Essentiel

¹ <https://tools.ietf.org/html/rfc6749>

² <http://www.passportjs.org/packages/passport-facebook/>

³ <https://github.com/passport/express-4.x-facebook-example>

⁴ <https://github.com/passport/express-4.x-facebook-example>

IX. Auto-évaluation

A. Exercice :

Le directeur de librairie qui vous a embauché pour faire l'API REST souhaite maintenant que les abonné(e)s puissent s'authentifier directement à partir du réseau social Facebook.

Question

En repartant de l'**application d'origine**¹, il vous demande de réaliser une simple page avec un lien cliquable qui permettrait aux utilisateurs de s'authentifier sur Facebook, puis de voir la liste des livres dont la boutique dispose en stock.

B. Test

Exercice : Quiz

Question 1

Quel en-tête HTTP doit envoyer le serveur pour que le navigateur affiche un formulaire d'authentification ?

- ☐ `Authorization-Form`
- ☐ `WWW-Authenticate Basic`
- ☐ `Form-Login Basic`

Question 2

Quel en-tête HTTP doit envoyer le client au serveur dans le cas d'une authentification HTTP Basic ?

- ☐ `Authorization`
- ☐ Un langage de programmation
- ☐ Une librairie qu'on installe dans notre projet node.js
- ☐ Un logiciel en ligne de commandes

Question 3

À quoi ressemblerait la valeur de cet en-tête ?

- ☐ `Basic am9objpzdXB1c1NlY3JldA==`
- ☐ `Auth login:mot_de_passe`
- ☐ `Basic login@mot_de_passe`
- ☐ `Bearer login:mot_de_passe`

Question 4

De combien d'éléments est composé un jeton JWT ?

- ☐ 1
- ☐ 2
- ☐ 3
- ☐ 4

¹ <https://github.com/mickaelandrieu/bookstore-node-rest-api>

Question 5

Que contient l'en-tête d'un jeton JWT ?

- ☐ La clé secrète et le type du jeton
- ☐ L'encodage à utiliser et la clé secrète
- ☐ La clé secrète et la durée de validité du jeton
- ☐ Le type de jeton et l'encodage à utiliser

Question 6

Que peut contenir l'enveloppe d'un jeton JWT ?

- ☐ La durée de validité du jeton
- ☐ Le nom de l'utilisateur
- ☐ Le rôle de l'utilisateur
- ☐ Le sujet du jeton

Question 7

Quels éléments sont indispensables pour créer la signature d'un jeton JWT ?

- ☐ L'en-tête encodé en Base 64
- ☐ L'enveloppe encodée en Base 64
- ☐ Une clé secrète uniquement connue du serveur
- ☐ Une méthode d'encodage

Question 8

Quelle réponse HTTP le serveur doit retourner si les identifiants passés sont invalides ?

- ☐ 401
- ☐ 400
- ☐ 200

Question 9

Quelle réponse HTTP le serveur doit-il retourner si une personne essaie d'accéder à une partie privée sans y être autorisée ?

- ☐ 401
- ☐ 500
- ☐ 200

Question 10

Quelle est la **première** chose à faire lorsque l'on souhaite intégrer un fournisseur d'authentification qui utilise OAuth 2 ?

- Installer les librairies **passport** et **moment**
- Créer une page avec un formulaire de connexion
- Créer un compte chez le fournisseur et déclarer l'URI de callback

Solutions des exercices

Exercice p. Solution n°1

Tout d'abord et à la racine du projet il fallait installer la librairie avec l'instruction suivante :

npm i express-basic-auth

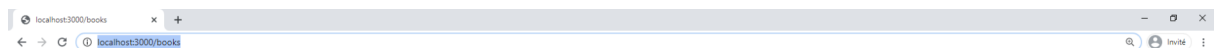
Ensuite, vous devez éditer le début du fichier index.js de cette façon :

```
1 const express = require('express');
2 const moment = require('moment');
3 const app = express()
4 const port = process.env.PORT || 3000;
5 const basicAuth = require('express-basic-auth')
6
7 app.use(basicAuth({
8   users: {
9     'johnathan@studi.com': 'sup3rS3cr3t',
10    'ieloushana@studi.com': 'YouKn0wN0thingJhon'
11  },
12  challenge: true
13}))
14
15 // Suite du fichier, à laisser à l'identique !
16 app.use(express.json())
```

Enfin, démarrer le serveur Node.js !

node index.js

Vous avez alors accès à l'API REST, et notamment à la liste des livres en accédant à l'**URL suivante**¹ telle que décrite dans le fichier *README.md* après authentification.



```
[{"title":"Book 1","author":"Jonathan","isbn":"0-7975-8110-3","availability":true,"return_date":null}, {"title":"Book 2","author":"Elise","isbn":"0-7975-8110-4","availability":false,"return_date":"2020-12-12T00:43:21.473Z"}, {"title":"Book 3","author":"Zineb","isbn":"0-7975-8110-5","availability":false,"return_date":null}]
```

Exercice p. Solution n°2

Une **solution est disponible sur Github**² et voici le code complet des fichiers index.js et package.json.

```
1 # index.js
2 const express = require('express');
3 const moment = require('moment');
4 const app = express()
5 const port = process.env.PORT || 3000;
6 const jwt = require('jsonwebtoken')
7 var e_jwt = require('express-jwt');
8
9 const SECRET = '6BF2C834E64251C2B4CD6492D3A1242561B8CEDB4ED3296341D6AA15E '
10
```

¹ <https://www.google.com/search?q=gerer-acces-api-accueil&oq=gerer-acces-api-accueil&aqs=chrome..69i57.1582j0j7&sourceid=chrome&ie=UTF-8>

² <https://github.com/mickaelandrieu/jwt-bookstore-node-rest-api>

```

11 app.use(express.json())
12
13 const users = [
14   {
15     "first_name": "Doe",
16     "last_name": "John",
17     "mail": "john.doe@studi.fr",
18     "password": "sEcr3t",
19     "books": [ ],
20   },
21   {
22     "first_name": "Doe",
23     "last_name": "Jane",
24     "mail": "jane.doe@studi.fr",
25     "password": "0neM0reThing",
26     "books": [ ],
27   },
28   {
29     "first_name": "Mickaël",
30     "last_name": "Andrieu",
31     "mail": "mickael.andrieu@exemple.fr",
32     "password": "12345678",
33     "books": ["0-7975-8110-3", "0-7975-8110-4"],
34   },
35 ]
36
37 const books = [
38   {
39     "title": "Book 1",
40     "author": "Jonathan",
41     "isbn": "0-7975-8110-3",
42     "availability": true,
43     "return_date": null
44   },
45   {
46     "title": "Book 2",
47     "author": "Elise",
48     "isbn": "0-7975-8110-4",
49     "availability": false,
50     "return_date": moment()
51   },
52   {
53     "title": "Book 3",
54     "author": "Zineb",
55     "isbn": "0-7975-8110-5",
56     "availability": false,
57     "return_date": null
58   },
59 ]
60
61 // Secure the application
62 app.use(e_jwt({ secret: SECRET, algorithms: ['HS256']})).unless({path: ['/login']}));
63
64 app.use(function (err, req, res, next) {
65   if (err.name === 'UnauthorizedError') {
66     res.status(401).send('Invalid token, access /login with mail and password in the Body to
67     authenticate.');
```

```
68   }
69 );
70
71 // Login
72 app.post('/login', (req, res) => {
73   // No Body
74   if (!req.body.mail || !req.body.password) {
75     return res.status(400).json({ message: 'Error. Please enter the correct email and
password' })
76   }
77
78   const user = users.find(u => u.mail === req.body.mail && u.password === req.body.password)
79
80   if (!user) {
81     return res.status(400).json({ message: 'Error. Wrong login or password' })
82   }
83
84   const token = jwt.sign({
85     username: user.mail
86   }, SECRET, { expiresIn: '3 hours' })
87
88   return res.json({ access_token: token })
89 })
90
91 app.get('/', (req, res) => {
92   res.json({ 'message': "Home, nothing interesting here." })
93 })
94
95 app.get('/books/', (req, res) => {
96   res.json(books)
97 })
98
99 app.get('/books/:isbn', (req, res) => {
100   const isbn = req.params.isbn
101   const book = books.find(book => book.isbn === isbn)
102
103   res.json(book)
104 })
105
106 app.post('/books', (req, res) => {
107   books.push(req.body)
108   res.status(200).json(books)
109 })
110
111 app.put('/books/:isbn', (req, res) => {
112   const isbn = req.params.isbn
113   let book = books.find(book => book.isbn === isbn)
114
115   // Book update
116   book.title = req.body.title
117   book.author = req.body.author
118   book.availability = req.body.availability
119   book.return_date = req.body.return_date
120
121   res.status(200).json(book)
122 })
123
124 app.delete('/books/:isbn', (req, res) => {
```

```

125     const isbn = req.params.isbn
126     const book = books.find(book => book.isbn === isbn)
127     books.splice(books.indexOf(book), 1)
128
129     res.json(books)
130 })
131
132 app.get('/users/', (req, res) => {
133     res.json(users)
134 })
135
136 app.get('/users/:mail', (req, res) => {
137     const mail = req.params.mail
138     const user = users.find(user => user.mail === mail)
139
140     res.json(user)
141 })
142
143 app.post('/users', (req, res) => {
144     users.push(req.body)
145     res.json(users)
146 })
147
148 app.put('/users/:mail', (req, res) => {
149     const mail = req.params.mail
150     const user = users.find(user => user.mail === mail)
151
152     user.first_name = req.body.first_name
153     user.last_name = req.body.last_name
154     user.mail = req.body.mail
155     user.books = req.body.books
156
157     res.json(user)
158 })
159
160 app.delete('/users/:mail', (req, res) => {
161     const mail = req.params.mail
162     const user = users.find(user => user.mail === mail)
163     users.splice(users.indexOf(user), 1)
164
165     res.json(users)
166 })
167
168 app.post('/borrow/:mail/:isbn', (req, res) => {
169     const mail = req.params.mail
170     const isbn = req.params.isbn
171
172     const user = users.find(user => user.mail === mail)
173     const book = books.find(book => book.isbn === isbn)
174
175     user.books.push(book.isbn)
176     book.availability = false
177     book.return_date = moment().add(14, 'days');
178
179     return res.json(book)
180 })
181
182 app.post('/bring-back/:mail/:isbn', (req, res) => {

```

```

183     const mail = req.params.mail
184     const isbn = req.params.isbn
185
186     const user = users.find(user => user.mail === mail)
187     const book = books.find(book => book.isbn === isbn)
188
189     user.books.splice(user.books.indexOf(book), 1)
190     book.availability = true
191     book.return_date = null;
192
193     return res.json(user)
194 })
195
196 app.listen(port, () => {
197   console.log('[Server] Started.')
198 })

```

Exercice p. Solution n°3

La solution a été décrite dans la vidéo précédente, la bonne URI de callback à déclarer dans Facebook est la suivante¹.

Exercice p. Solution n°4

Dans ce projet, il nous faut implémenter le système d'authentification OAuth à l'aide de la librairie **passport** et adapter la page d'action pour qu'elle affiche tous les livres disponibles.

Ci-dessous le code fonctionnel de ce projet ainsi que le fichier package.json qui contient la liste des dépendances.

```

1 # index.js
2 const express = require('express');
3 const moment = require('moment');
4 const app = express();
5 const port = process.env.PORT || 3000;
6 const session = require('express-session');
7 const passport = require('passport');
8 const FacebookStrategy = require('passport-facebook').Strategy;
9
10 const SECRET = 'S3cr3tK3yD0ntSh4r31t'
11 const FACEBOOK_ID = '2436556759987583'
12 const FACEBOOK_SECRET = 'be251a8b7a12021136ec9d42c2b7d849'
13
14 app.use(express.json())
15 app.use(session({
16   resave: false,
17   saveUninitialized: true,
18   secret: SECRET
19 }));
20
21 app.use(passport.initialize());
22 app.use(passport.session());
23
24 passport.serializeUser(function (user, cb) {
25   cb(null, user);

```

¹ <http://localhost:8000/return>

```

26 });
27
28 passport.deserializeUser(function (obj, cb) {
29   cb(null, obj);
30 });
31
32 passport.use(new FacebookStrategy({
33   clientID: FACEBOOK_ID,
34   clientSecret: FACEBOOK_SECRET,
35   callbackURL: 'http://localhost:3000/oauth-redirect'
36 }, function (accessToken, refreshToken, profile, done) {
37   return done(null, profile);
38 }
39 ));
40
41 app.get('/', function (req, res) {
42   res.send('<a href="/login">Login</a>')
43 });
44
45 app.get('/login', passport.authenticate('facebook', {
46   scope: ['public_profile']
47 }));
48
49 app.get('/oauth-redirect',
50   passport.authenticate('facebook', {
51     successRedirect: '/action',
52     failureRedirect: '/error'
53   })
54 );
55
56 app.get('/action', isLoggedIn, function (req, res) {
57   // register a user.
58
59   res.json(req.user)
60 });
61
62 /**
63  * Demo Data START
64  */
65 const users = [
66   {
67     "first_name": "Doe",
68     "last_name": "John",
69     "mail": "john.doe@studi.fr",
70     "books": [ ],
71   },
72   {
73     "first_name": "Doe",
74     "last_name": "Jane",
75     "mail": "jane.doe@studi.fr",
76     "books": [ ],
77   },
78   {
79     "first_name": "Mickaël",
80     "last_name": "Andrieu",
81     "mail": "mickael.andrieu@exemple.fr",
82     "books": ["0-7975-8110-3", "0-7975-8110-4"],
83   },

```

```
84 ]
85
86 const books = [
87   {
88     "title": "Book 1",
89     "author": "Jonathan",
90     "isbn": "0-7975-8110-3",
91     "availability": true,
92     "return_date": null
93   },
94   {
95     "title": "Book 2",
96     "author": "Elise",
97     "isbn": "0-7975-8110-4",
98     "availability": false,
99     "return_date": moment()
100  },
101  {
102    "title": "Book 3",
103    "author": "Zineb",
104    "isbn": "0-7975-8110-5",
105    "availability": false,
106    "return_date": null
107  },
108 ]
109
110 /**
111  * Demo Data END
112  */
113
114 /**
115  * Books CRUD
116  */
117 app.get('/', isLoggedIn, (req, res) => {
118   res.json({'message': "Home, nothing interesting here."})
119 })
120
121 app.get('/books/', isLoggedIn, (req, res) => {
122   res.json(books)
123 })
124
125 app.get('/books/:isbn', isLoggedIn, (req, res) => {
126   const isbn = req.params.isbn
127   const book = books.find(book => book.isbn === isbn)
128
129   res.json(book)
130 })
131
132 app.post('/books', isLoggedIn, (req, res) => {
133   books.push(req.body)
134   res.status(200).json(books)
135 })
136
137 app.put('/books/:isbn', isLoggedIn, (req, res) => {
138   const isbn = req.params.isbn
139   let book = books.find(book => book.isbn === isbn)
140
141   // Book update
```

```

142   book.title = req.body.title
143   book.author = req.body.author
144   book.availability = req.body.availability
145   book.return_date = req.body.return_date
146
147   res.status(200).json(book)
148 })
149
150 app.delete('/books/:isbn', isLoggedIn, (req, res) => {
151   const isbn = req.params.isbn
152   const book = books.find(book => book.isbn === isbn)
153   books.splice(books.indexOf(book), 1)
154
155   res.json(books)
156 })
157
158 /**
159 * Users CRUD
160 */
161 app.get('/users/', isLoggedIn, (req, res) => {
162   res.json(users)
163 })
164
165 app.get('/users/:mail', isLoggedIn, (req, res) => {
166   const mail = req.params.mail
167   const user = users.find(user => user.mail === mail)
168
169   res.json(user)
170 })
171
172 app.post('/users', isLoggedIn, (req, res) => {
173   users.push(req.body)
174   res.json(users)
175 })
176
177 app.put('/users/:mail', isLoggedIn, (req, res) => {
178   const mail = req.params.mail
179   const user = users.find(user => user.mail === mail)
180
181   // Update of the user
182   user.first_name = req.body.first_name
183   user.last_name = req.body.last_name
184   user.mail = req.body.mail
185   user.books = req.body.books
186
187   res.json(user)
188 })
189
190 app.delete('/users/:mail', isLoggedIn, (req, res) => {
191   const mail = req.params.mail
192   const user = users.find(user => user.mail === mail)
193   users.splice(users.indexOf(user), 1)
194
195   res.json(users)
196 })
197
198 /**
199 * Book Store management

```

```

200 */
201
202 app.post('/borrow/:mail/:isbn', isLoggedIn, (req, res) => {
203   const mail = req.params.mail
204   const isbn = req.params.isbn
205
206   const user = users.find(user => user.mail === mail)
207   const book = books.find(book => book.isbn === isbn)
208
209   user.books.push(book.isbn)
210   book.availability = false
211   book.return_date = moment().add(14, 'days');
212
213   return res.json(book)
214 })
215
216 app.post('/bring-back/:mail/:isbn', isLoggedIn, (req, res) => {
217   const mail = req.params.mail
218   const isbn = req.params.isbn
219
220   const user = users.find(user => user.mail === mail)
221   const book = books.find(book => book.isbn === isbn)
222
223   user.books.splice(user.books.indexOf(book), 1)
224   book.availability = true
225   book.return_date = null;
226
227   return res.json(user)
228 })
229
230 app.get('/error', isLoggedIn, function (req, res) {
231   res.send('Login error');
232 });
233
234 app.get('/logout', function (req, res) {
235   req.logout();
236   res.redirect('/');
237 });
238
239 function isLoggedIn(req, res, next) {
240   if (req.isAuthenticated())
241     return next();
242   res.redirect('/');
243 }
244
245 app.listen(port, () => {
246   console.log('[OAuth BookStore Server] Started.')
247 })

```

```

1 {
2   "name": "oauth2-secure-api",
3   "version": "1.0.0",
4   "description": "",
5   "main": "index.js",
6   "scripts": {
7     "start": "node index.js"
8   },
9   "author": "Vous<votre_mail@exemple.com>",
10  "license": "ISC",

```

```

11  "dependencies": {
12    "express": "^4.17.1",
13    "express-session": "^1.17.1",
14    "moment": "^2.29.1",
15    "passport": "^0.4.1",
16    "passport-facebook": "^3.0.0"
17  }
18 }

```

Exercice p. 22 Solution n°5

Question 1

Quel en-tête HTTP doit envoyer le serveur pour que le navigateur affiche un formulaire d'authentification ?

- ☐ Authorization-Form
- ☒ WWW-Authenticate Basic
- ☐ Form-Login Basic



Quand le navigateur reçoit cet en-tête en réception de sa requête alors il affiche un formulaire de connexion qui sera renvoyé au serveur.

Question 2

Quel en-tête HTTP doit envoyer le client au serveur dans le cas d'une authentification HTTP Basic ?

- ☒ Authorization
- ☐ Un langage de programmation
- ☐ Une librairie qu'on installe dans notre projet node.js
- ☐ Un logiciel en ligne de commandes



Tous les systèmes d'authentification envoient cet en-tête mais la valeur diffère en fonction du type d'authentification : basic pour HTTP Basic, et Bearer pour les systèmes à token comme JWT ou OAuth 2.

Question 3

À quoi ressemblerait la valeur de cet en-tête ?

- ☒ Basic am9objpzdXB1c1NlY3JldA==
- ☐ Auth login:mot_de_passe
- ☐ Basic login@mot_de_passe
- ☐ Bearer login:mot_de_passe



Cet en-tête commence par Basic suivi des identifiants de connexion qui sont encodés pour des raisons de sécurité.

Question 4

De combien d'éléments est composé un jeton JWT ?

- ☐ 1
- ☐ 2

☒ 3

☐ 4

 Le jeton JWT est composé de 3 éléments : un en-tête, une enveloppe et une signature.

Question 5

Que contient l'en-tête d'un jeton JWT ?

☐ La clé secrète et le type du jeton

☐ L'encodage à utiliser et la clé secrète

☐ La clé secrète et la durée de validité du jeton

☒ Le type de jeton et l'encodage à utiliser

 Selon la spécification JWT, l'en-tête contient le type de jeton (« *JWT* ») et l'encodage utilisé (souvent: « *HS256* »).

Question 6

Que peut contenir l'enveloppe d'un jeton JWT ?

☒ La durée de validité du jeton

☒ Le nom de l'utilisateur

☒ Le rôle de l'utilisateur

☒ Le sujet du jeton

 Si certains éléments sont décrits dans la spécification JWT sous forme de recommandations, dans la pratique on peut mettre dans l'enveloppe (ou payload) du jeton tout ce que l'on souhaite !

Question 7

Quels éléments sont indispensables pour créer la signature d'un jeton JWT ?

☒ L'en-tête encodé en Base 64

☒ L'enveloppe encodée en Base 64

☒ Une clé secrète uniquement connue du serveur

☒ Une méthode d'encodage

 Nous avons besoin de tous ces éléments pour créer la signature, il est possible de créer la signature d'un jeton manuellement sur le site de jwt¹.

Question 8

Quelle réponse HTTP le serveur doit retourner si les identifiants passés sont invalides ?

☐ 401

☒ 400

☐ 200

¹ <https://jwt.io/#debugger-io>

🔍 Par convention, vous devez renvoyer une réponse avec le code d'erreur 400¹ (« *Bad Request* ») quand les identifiants passés ne permettent pas d'authentifier un utilisateur.

Question 9

Quelle réponse HTTP le serveur doit-il retourner si une personne essaie d'accéder à une partie privée sans y être autorisée ?

- ☒ 401
- ☐ 500
- ☐ 200

🔍 Par convention, vous devez renvoyer une réponse avec le code d'erreur 401 (« *Unauthorized* ») quand un utilisateur cherche à accéder à une partie protégée sans en avoir le droit.

Question 10

Quelle est la **première** chose à faire lorsque l'on souhaite intégrer un fournisseur d'authentification qui utilise OAuth 2 ?

- ☐ Installer les librairies **passport** et **moment**
- ☐ Créer une page avec un formulaire de connexion
- ☒ Créer un compte chez le fournisseur et déclarer l'URI de callback

🔍 Le principe même d'OAuth, c'est que vous déléguez la responsabilité de l'authentification à un fournisseur. Il faudra donc que vous authentifiez votre application auprès de ce fournisseur pour lui indiquer sur quel URI il doit vous renvoyer un jeton avec les informations demandées.

¹ code d'erreur 400